

PART 1 – Vulnerability Assessment (Analysis Phase)

This section presents the identified vulnerabilities and security weaknesses found in the Student Management System developed using PHP and MySQL. The assessment focuses on authentication security, database integrity, session management, and input validation.

A. Application-Level Vulnerabilities

1. Plaintext Password Authentication

Risk Level: High

Description

The system originally authenticated users using direct password comparison:

```
if($password === $user['password'])
```

Passwords were stored in plaintext inside the database.

Risk

If the database is compromised, attackers can immediately view all user passwords.

Security Improvement

Password validation was improved and secure password handling mechanisms were recommended using:

- `password_hash()`
 - `password_verify()`
-

2. Duplicate Student ID Vulnerability

Risk Level: Medium

Description

The system originally allowed multiple student records with the same Student ID.

Risk

This can cause:

- duplicate records
- database redundancy
- inconsistent student information

Security Improvement

Duplicate checking was implemented before inserting student records.

Example:

```
$check = $conn->prepare("SELECT student_id FROM students WHERE student_id=?");
```

A UNIQUE database constraint was also recommended.

3. Weak Email Validation Handling

Risk Level: Low

Description

The system originally terminated the page immediately when invalid email input was detected.

Example:

```
die("Invalid email format.");
```

Risk

Poor user experience and improper validation flow.

Security Improvement

Validation handling was improved using alert messages and controlled page redirection.

4. Weak Session Handling

Risk Level: Medium

Description

The logout mechanism originally used only basic session destruction.

Risk

Possible session reuse and session fixation issues.

Security Improvement

Improved session cleanup was implemented using:

```
session_unset();  
session_destroy();
```

5. Missing Database Constraints

Risk Level: Medium

Description

The database initially had no UNIQUE constraint for Student IDs.

Risk

This allowed duplicate entries and reduced database integrity.

Security Improvement

Database normalization and constraints were recommended using:

```
ALTER TABLE students ADD UNIQUE(student_id);
```

6. Missing Access Control Validation

Risk Level: Medium

Description

Administrative pages could be accessed without proper session validation.

Risk

Unauthorized users may access protected pages and manipulate records.

Security Improvement

Session validation was implemented on protected pages.

Example:

```
if(!isset($_SESSION['user'])){  
    header("Location: login.php");  
}
```

B. Database-Level Issues

- Database Issue
- Missing Normalization
- Data Redundancy
- Missing Foreign Keys
- No Password Encryption
- No Backup System
- Weak Data Integrity

PART 2 – Debugging and Security Improvements

1. Duplicate Student ID Prevention

Problem

The original system allowed multiple students to use the same Student ID.

Improvement Applied

A duplicate checking query was added before inserting a new student.

Code Implemented

```
$check = $conn->prepare("SELECT student_id FROM students WHERE student_id=?");
$check->bind_param("s", $student_id);
$check->execute();
$check->store_result();

if($check->num_rows > 0){
    die("Student ID already exists!");
}
```

Result

- Prevented duplicate student records
 - Improved database consistency
-

2. Input Validation Improvement

Problem

The system originally accepted invalid inputs without proper validation.

Improvement Applied

Validation checks were added for:

- empty fields
- invalid email formats

Code Implemented

```
if(empty($student_id) || empty($fullname) || empty($email)){
    die("Required fields missing.");
}
```

Email Validation

```
if(!filter_var($email, FILTER_VALIDATE_EMAIL)){  
    echo "<script>alert('Invalid email format'); window.history.back();</script>";  
    exit();  
}
```

Result

- Reduced invalid data submissions
 - Improved form validation handling
-

3. Prepared Statements

Problem

Direct SQL queries may expose the system to SQL Injection vulnerabilities.

Improvement Applied

Prepared statements were used in database queries.

Code Implemented

```
$stmt = $conn->prepare("  
    INSERT INTO students (student_id, fullname, email, course, course_description)  
    VALUES (?, ?, ?, ?, ?)  
");
```

Delete Query

```
$stmt = $conn->prepare("DELETE FROM students WHERE id=?");
```

Result

- Improved database query security
 - Reduced SQL Injection risk
-

4. Output Sanitization

Problem

Displaying raw user input may expose the system to XSS attacks.

Improvement Applied

The system used `htmlspecialchars()` when displaying student records.

Code Implemented

```
<?php echo htmlspecialchars($row['fullname']); ?>
```

Result

- Improved output security
 - Prevented script injection through displayed data
-

5. Session Validation

Problem

Users may directly access protected pages without authentication.

Improvement Applied

Session validation was added to protected pages.

Code Implemented

```
if(!isset($_SESSION['user'])){  
    header("Location: login.php");  
    exit();  
}
```

Result

- Restricted unauthorized access
 - Improved access control
-

6. Improved Logout Handling

Problem

The logout system originally used only basic session destruction.

Improvement Applied

Additional session cleanup functions were added.

Code Implemented

```
$_SESSION = [];  
session_unset();  
session_destroy();
```

Result

- Improved session cleanup
 - Reduced session reuse risks
-

7. CSS and User Interface Improvements

Problem

The original interface was minimal and inconsistent.

Improvement Applied

The `style.css` file was improved to provide:

- cleaner layout

- improved forms
- better table styling
- improved button design

<https://github.com/gdoromal510892/Labexam2Dorms.git>

PART 3 – Documentation

The Student Management System was checked and tested to find problems and weaknesses in the system. During the testing, several issues were found in the login system, data validation, database design, and session handling. These issues could affect the security and proper working of the system if not fixed.

One of the main problems found was the use of plaintext passwords. The system originally checks the password directly using simple comparison. This means the password is stored as plain text in the database, which is not safe because anyone who gets access to the database can see all user passwords.

Another issue was the duplicate Student ID problem. The system allows adding students with the same Student ID because there was no checking before inserting data. This can cause repeated records and confusion in the database.

There was also a problem with email validation handling. When the user enters an invalid email, the system stops immediately using a simple `die()` message. This works, but it is not user-friendly because it suddenly stops the process instead of giving a proper warning.

Session handling was also basic. The logout function only destroys the session, but it does not fully clear all session data properly in a secure way. This may cause small security issues in some cases.

Another issue is that the database has no proper constraints like UNIQUE keys for Student ID. Because of this, duplicate records can still be added if validation fails in the code.

To fix these problems, several improvements were made in the system. A duplicate check was added before inserting a student so that the same Student ID cannot be added twice. Input validation was also improved by checking empty fields and making sure the email format is correct before saving the data.

Prepared statements were used in database queries like insert and delete to make the system more secure and to prevent SQL injection. Output was also improved using `htmlspecialchars()` so that data shown on the dashboard is safer and cannot run unwanted code.

Session checking was added so only logged-in users can access pages like the dashboard and add student page. The logout system was also improved to properly clear session data.

The design of the system was also improved by updating the CSS. The form, table, and buttons are now cleaner and easier to use.

Lastly, a backup plan was suggested. The database should be backed up daily using phpMyAdmin or export tools, and the backup files should be saved in a safe place like cloud storage or external drive. This is important in case the system data is lost or damaged.

Overall, the system is now more stable, more secure, and easier to use after the improvements were applied.